Informe TIC caso 3

Tecnología e Infraestructura de Cómputo

Universidad de los Andes

2026-2

Sara Gomez Florez 202513246

Julian Andres Guzman 202414714

4 de noviembre 2026

Preguntas

2. Gestor de solicitudes

a) Si el número de MVs en espera es igual o superior al umbral, el gestor debe esperar a que todas las solicitudes pendientes se procesen (no permite el ingreso de solicitudes adicionales hasta terminar con las pendientes). ¿Qué problema resuelve esta manera de procesar solicitudes?

El problema principal es que si hay una MV que necesita de un gran espacio de recursos para su ejecución y ya hay bastantes MV ejecutándose, si se siguen permitiendo entrar el poco espacio que se libere va a ser absorbido por esas MVs que requieren menos recursos y la MV que necesita de grandes recursos pueda quedarse esperando indefinidamente que haya espacio suficiente. Esto se resuelve si se espera que las solicitudes pendientes se procesen, lo que conlleva a que las MVs que necesiten un mayor uso de recurso puedan tener suficientes recursos y procesarse porque ya no hay nadie que los ocupe. A este problema se le llama inanición o postergación indefinida.

3. MV

a) Una máquina reserva los recursos que necesita -uno a uno- y siempre en el orden establecido: procesador, memoria y disco. ¿Por qué es importante que todas sigan el mismo orden de solicitud?

Es importante ya que si existe el caso de que dos máquinas virtuales necesiten un recurso y lo pidan de manera aleatoria, se pueden quedar esperando ese recurso. Por ejemplo, una MV que necesite memoria y otra que tenga la memoria restante pero necesite la cantidad de procesadores que tiene la MV. Esto hace que se queden esperando unas con otras para siempre (o hasta timeout).

b) Si, después de haber recibido uno o más recursos, encuentra que el siguiente recurso que necesita no está disponible, debe liberar inmediatamente todos los recursos que había reservado previamente. Por ejemplo, si una máquina ya recibió procesadores y memoria, pero no puede obtener el disco requerido, deberá devolver tantos los procesadores como la memoria. ¿Por qué es importante liberar los recursos asignados cuando no se logra completar la reserva?

Para ejecutar una máquina virtual se necesitan los tres recursos: procesador, memoria y disco. Si solo tiene 1 o 2, no puede hacer nada útil con ellos. Si se queda con esos recursos mientras espera que se liberen el resto, los recursos quedan secuestrados, osea que la MV no los está usando por esperar y nadie más puede usarlos por que esa misma MV los tiene retenidos esperando el recurso faltante.
Al liberar los recursos asignados de la VM se imposibilita la ocurrencia de un deadlock pues, o una VM lo tiene todo o no tiene nada. Además, al tomar estas medidas el sistema nunca queda en un estado intermedio donde una máquina tiene recursos a medias. Eso facilita razonar sobre el estado del sistema y evita bloqueos en cadena.

c) Cuando termine su ejecución, la máquina deberá liberar todos los recursos que tenía asignados y despertar otra máquina para que use los recursos. debería despertar una sola o todos? ¿por qué?
R/Hay que despertar a todas porque cada máquina espera recursos distintos en distintas cantidades. Si se despierta solo una, puede tocarle a una que no puede avanzar, se vuelve a dormir y el aviso se pierde, mientras otra que sí podía avanzar se queda dormida. Al despertar a todas, cada una revisa si le alcanza y las que pueden avanzan, las demás vuelven a esperar.

Funcionamiento Programa

Usuario

Usuario es un thread que tiene tres atributos, numSolicitudes, nUsuario y el buzón recibido por parámetro. Su función es generar solicitudes que se depositan en el buzón de entrada.

Atributos

int numSolicitudes	Es el número de solicitudes que va a generar ese usuario.
int nUsuario	Es el número de usuario
Buzon buzon	Es el buzón del sistema

Métodos

- El constructor, que recibe por parámetro nUsuario, numSolicitudes y buzon.
- Como es un thread, tiene un run(). Este run genera un ciclo for donde se genera la cantidad de solicitudes que tiene que crear el usuario. Solicitud es una clase que recibe su id como parámetro. Cuando la crea, llama al método de buzon agregarSolicitud(). Cuando termina de agregar todas sus solicitudes, le dice al buzón que ya terminó disminuyendo el contador de usuarios.

Solicitud

Una solicitud pide la reserva de recursos para generar una máquina virtual.

Atributos

String nombre	Tiene dos partes: el id del usuario que generó la solicitud y una parte secuencial. Es decir, si el usuario uno está creando la solicitud tres el id de la solicitud es: “U1-S3”
int numeroProcesadores	El número de procesadores que necesita la solicitud (depende de lo configurados en

int numMemoria	La cantidad de memoria que necesita la solicitud.
int espacioDisco	La cantidad de espacio disco que se necesita en la solicitud.

Configuracion

Aquí está el main, que funciona de la siguiente forma:
lee un archivo txt y saca los parámetros que necesita para inicializar el sistema. Inicializa el buzon, la zona de espera con sus atributos y los controladores.

Luego crea los threads del sistema, las MVs, los usuarios y los gestores.

Buzón

Atributos

int cantUsuarios	Se inicializa con la cantidad de usuarios que hay. Esto porque si es cero, puede que no se hayan cargado solicitudes y el gestor decida terminar antes de tiempo (porque no se sabe el orden de ejecución de los procesos, si usuario o gestor primero).
ArrayList solicitudes	Las solicitudes en cola enviadas por los usuarios.

Métodos

- El constructor: Recibe la cantidad de usuarios pasado por el main.
- disminuirUsuario(): Este método lo llama usuario para indicar que ya terminó de enviar sus solicitudes. Es synchronized porque varios threads pueden manipularlo, y es solo uno porque es static (en el main)
- noHaySolicitudesUsuarios(): Este método devuelve si no hay solicitudes pero sí usuarios (porque puede haber el caso de que no haya solicitudes pero todavía usuarios que pueden mandar solicitudes)
- haySolicitudesUsuarios(): Este método devuelve si hay solicitudes y usuarios.
- haySolicitudes(): Este método devuelve true si hay solicitudes. Se utiliza para verificar que todavia haya solicitudes porque puede haber el caso de que no haya usuarios pero si solicitudes por procesar.
- isBuzonTerminado(): Devuelve true si ya no hay solicitudes y no hay usuarios.
- agregarSolicitud(): Recibe la solicitud enviada por el usuario y revisa si el buzón de solicitudes no ha llegado a su capacidad. Si llegó, espera pasivamente. Sino, lo agrega a las solicitudes y le notifica al gestor que agregó una solicitud por si está esperando.

- sacarSolicitud(): Este método lo llama el gestor, si hay solicitudes la saca y notifica al usuario que hay espacio. Si no puede, devuelve null.

Todo esto lleva synchronized porque son métodos que son compartidos entre threads y que es importante la persistencia de los elementos que maneja.

Gestor

Atributos

Buzon buzon	Es el buzón compartido por todos los threads enviado por main.
ZonaEspera zonaEspera	Es la zona de espera compartida por todos los threads, enviada por parámetro a través de main.

Métodos

- constructor: recibe los parámetros enviados por el main (Configuracion)
- run(): El proceso de gestor es el siguiente:

1. Primero, sino hay solicitudes pero si usuarios o hay solicitudes mira que:
2. si no hay solicitudes pero si usuarios, espera semi-pasivamente. Esto porque puede que no haya solicitudes y no usuarios, y puede quedarse esperando.
3. Luego, mientras haya solicitudes por procesar verifica:
4. Si la zona de espera no supera el umbral propuesto, la saca del buzón. Sí sí se puede sacar, se mira que no retorne null y se crea la máquina virtual y se ejecuta.
5. En cambio, si no se pudo agregar, significa que estaba llena. Espera.
6. Si sí supera el umbral, mientras haya máquinas virtuales en el umbral, espera (con yield)

ZonaEspera

La zona de espera es donde las MVs que no lograron conseguir tres recursos se duermen. No es un thread, es un objeto compartido entre las MVs. No tiene ningun limite en cuantas MVs puede tener y el umbral solo lo usa el gestor para decidir si sigue sacando solicitudes.
Las MVs se duermen sobre este mismo objeto, haciendo zonaEspera.wait() cuando no les alcanzan los recursos, y la MV que termina su ejecución hace zonaEspera.notifyAll().

ArrayList<MaquinaVirtual> maquinasVirtuales	Es donde están las MVs durmiendo mientras esperan que se liberen recursos. Una MV se agrega justo antes de dormirse y se saca cuando se despiertan para volver a intentar la reserva.
int umbral	Se toma de Configuracion.tam2 al crear la zona. Es la cantidad de MVs en espera a partir de la cual el gestor deja de sacar solicitudes del buzón hasta que se atiendan todas las pendientes.

Métodos

- superaUmbral() synchronized: retorna true si hay igual o más MVs en espera que el umbral. Lo usa el gestor antes de sacar una solicitud del buzón.
- hayMaquinasVirtuales() synchronized: retorna true si todavía hay MVs en la lista. Cuando se supera el umbral, el gestor lo llama en su espera semiactiva con yield hasta que todas las pendientes consigan sus recursos.
- agregarMaquinaVirtual(MaquinaVirtual) synchronized: agrega la MV a la lista. Lo llama la MV justo antes de hacer wait().
- sacarMaquinaVirtual(MaquinaVirtual) synchronized: saca la MV de la lista. Lo llama la MV apenas despierta, antes de volver a intentar la reserva.

Todos los métodos son synchronized porque el ArrayList no es seguro para varios threads: si un gestor consultara el tamaño de la lista mientras una MV la está modificando, podría leer un valor distinto.

MaquinaVirtual

La máquina virtual es un thread que crea el gestor por cada solicitud que puede procesar, con new MaquinaVirtual(solicitud) y después start(). Primero se crea la MV y después ella misma pide los recursos en su run(). Así, quien reserva y quien se duerme es el thread de la MV si es que no hay recursos y no el del gestor, que queda libre para seguir sacando solicitudes del buzón.

Atributos

static ZonaEspera zonaEspera	La zona de espera, que la misma para todas las MVs. Es static porque el gestor solo le pasa la solicitud al crear la MV, entonces el main la asigna una sola vez con configurar().
static ControladorProcesadores ctrlProcesador static controladorMemoria ctrlMemoria static controladorAlmacenamiento ctrlAlmacenamiento	Los tres controladores de recursos son static por la misma razón, los recursos son del sistema y todas las MVs deben usar los mismos controladores.
static int activas	Cuántas MVs se han creado y todavía no han terminado. Lo usa el main para esperar a que terminen todas antes de revisar el estado final.
Solicitud solicitud	La solicitud con la que se creó la MV. De aquí saca cuántos procesadores, cuánta memoria y cuánto disco necesita.
String id	Identificador de la MV: "MV-" más el nombre de la solicitud. Con este nombre los controladores registran qué recursos tiene cada MV.

Métodos

- configurar(zona, procesador, memoria, almacenamiento) static: lo llama el main una sola vez, antes de arrancar usuarios y gestores, para darle a todas las MVs la zona de espera y los tres controladores.
- MaquinaVirtual(Solicitud solicitud): guarda la solicitud, arma el id y suma 1 a activas con sumarActiva(). Se cuenta en el constructor el cual lo ejecuta el gestor y no en run(), para que el main nunca vea 0 MVs activas mientras hay MVs creadas que todavía no han arrancado.
- run(): tiene tres pasos:
  1. Pide los recursos con reservarTodo() dentro de synchronized (zonaEspera). Mientras no lo logre, se agrega a la zona de espera, hace zonaEspera.wait() y, cuando la despiertan, se saca de la lista y vuelve a intentar desde los procesadores.
  2. Cuando tiene los tres recursos, se ejecuta durante un tiempo aleatorio entre 4000 y 40000 ms con Thread.sleep().
  3. Libera los tres recursos y hace zonaEspera.notifyAll() para despertar a todas las MVs que están esperando, también dentro de synchronized (zonaEspera). Al final, en el finally, resta 1 a activas con restarActiva().
- reservarTodo(): intenta reservar en el orden procesadores, memoria y almacenamiento. Si alguno falla, libera los que ya había conseguido y retorna false. Si consigue los tres, retorna true.
- sumarActiva() y restarActiva() static synchronized: suman o restan 1 a activas. restarActiva() además hace MaquinaVirtual.class.notifyAll() para avisarle al main. Como son static synchronized, el candado que usan es el de la clase (MaquinaVirtual.class) y no el de una MV en particular.
- esperarTodas() static synchronized: la llama el main al final. Mientras activas sea mayor que 0 hace MaquinaVirtual.class.wait(), y sale cuando terminaron todas las MVs.

ControladorProcesadores

Es un objeto pasivo que lleva el registro de los procesadores del sistema. Sus métodos son synchronized, así que solo una MV a la vez puede reservar o liberar procesadores. Ningún método hace esperar al thread, si no alcanzan, solo retorna false, y la espera se hace en la zona de espera.

Atributos

String[] asignados	Tiene una casilla por procesador. Si la casilla está en null, el procesador está libre; si no, tiene el id de la MV que lo tiene. Así se sabe cuáles procesadores están asignados, cuáles están libres y qué MV tiene cada uno.
int libres	Cuántos procesadores están libres. Sirve para saber rápido si alcanzan sin recorrer todo el arreglo.

Métodos

- reservar(String idMV, int cantidad) synchronized: si no hay suficientes procesadores libres retorna false sin asignar ninguno (todo o nada). Si alcanzan, recorre el arreglo y pone el id de la MV en las primeras casillas libres hasta completar la cantidad, y resta esa cantidad de libres.
- liberar(String idMV) synchronized: recorre el arreglo y vuelve a poner en null todas las casillas que tenían el id de esa MV. Suma a libres los procesadores que devolvió y retorna cuántos fueron.
- duenoDe(int procesador) y procesadoresDe(String idMV) synchronized: son consultas sobre el registro: qué MV tiene un procesador y qué procesadores tiene una MV.
- getLibres(), getTotal(), todoLibre() y estado(): se usan al final del programa para verificar que todos los procesadores quedaron libres y para imprimir el estado del controlador.

controladorMemoria

Lleva el registro de la RAM que se le asigna a cada MV, dividida en páginas del tamaño que dice la configuración. Funciona como el de procesadores, es un objeto pasivo, sus métodos son synchronized y reservar todo o nada. La diferencia es que la memoria puede tener millones de páginas, así que en vez de un arreglo con una casilla por página usa un BitSet, que gasta un bit por página, y un mapa con los tramos de páginas que tiene cada MV.

Atributos

static final long KIB_POR_GIB	Cuántos KiB tiene un GiB: 1024 × 1024 = 1.048.576. Sirve para pasar las solicitudes de GiB a KiB.
int tamPagina	Tamaño de cada página en KiB. Viene de la configuración.
int totalPaginas	Cuántas páginas hay en total y es la capacidad de memoria en KiB dividida entre el tamaño de página.
BitSet paginasOcupadas	Un bit por página, 1 si la página está asignada y 0 si está libre.
Map<String, List<Tramo>> asignaciones	Para cada MV, identificada por su id, la lista de tramos de páginas que tiene. Así se sabe qué MV tiene cada página.
int paginasLibres	Cuántas páginas están libres.
Tramo (clase interna)	Un grupo de páginas seguidas que tiene una MV, con su página de inicio y su cantidad.

Métodos

- aPaginas(int gib): convierte los GiB que pide la MV a páginas. Primero pasa los GiB a KiB y luego divide entre el tamaño de página, redondeando hacia arriba para no darle a la MV menos memoria de la que pidió.
- reservar(String idMV, int gib) synchronized, calcula cuántas páginas necesita la MV. Si no alcanzan, retorna false sin asignar nada. Si alcanzan, busca huecos de páginas libres en el BitSet, los marca como ocupados y anota cada tramo en la lista de la MV, hasta completar las páginas que necesita.
- liberar(String idMV) synchronized: saca del mapa los tramos de la MV, pone en 0 sus bits en el BitSet y suma esas páginas a paginasLibres. Retorna cuántas páginas liberó.
- duenoDe(int pagina) y asignacionDe(String idMV) synchronized: consultas sobre el registro: qué MV tiene una página y qué tramos de páginas tiene una MV.
- getLibres(), getTotal(), todoLibre() y estado(): se usan al final para verificar que toda la memoria quedó libre y para imprimir el estado.

controladorAlmacenamiento

Funciona igual que el controlador de memoria, pero con el disco dividido en bloques del tamaño que dicta la configuración. Sus atributos son los mismos, cambiando páginas por bloques, o sea, tamBloque, totalBloques, bloquesOcupados, asignaciones y bloquesLibres. Sus métodos también son los mismos, es decir, aBloques(int gib) convierte los GiB pedidos a bloques redondeando hacia arriba, y reservar, liberar, duenoDe, asignacionDe, getLibres, getTotal, todoLibre y estado hacen lo mismo que en memoria, pero con bloques.